

INGENIERÍA DEL SOFTWARE 3 · UCC 2026

Trabajo Práctico 01

Git colaborativo

Alumno	< completar: nombre y apellido >
Matrícula	2319274
Usuario GitHub	marcosdon28
Repositorio	github.com/marcosdon28/IngeSoft3-2319274
Versión entregada	v1.0.0
Fecha	26/08/2026
Riel	GitHub (canónico)

1. Objetivo y qué se evalúa

El TP1 pide poner a funcionar —y poder defender— el flujo con el que un equipo integra código: ramas cortas que entran por Pull Request, revisión antes de integrar, protecciones sobre `main` y versionado de la entrega. Antes de escribir una línea de código de producción se deja armado *cómo* va a entrar ese código.

Este repositorio no es sólo el del TP1: es **el repositorio del semestre**. La app elegida en el TP2 entra acá mismo, y cada TP siguiente le suma una capa. Por eso las decisiones de este trabajo —protecciones, convención de ramas, estrategia de merge— condicionan todo lo que viene.

Tareas exigidas por el enunciado

Requisito	Estado
Repositorio público con <code>.gitignore</code> en la raíz	✓ Cumplido
<code>main</code> protegida: imposible pushear directo, sin bypass ni para el dueño	✓ Cumplido y verificado
Al menos 2 PRs mergeados, uno con conflicto resuelto	✓ 3 PRs; el #3 con conflicto
Tag <code>v1.0.0</code> con su release publicada	✓ Cumplido
<code>decisiones.md</code> y <code>evidencias.md</code> en la raíz	✓ Cumplido, entraron por PR

La regla innegociable de la materia: «si no lo podés explicar, no lo aprobás — aunque funcione». La defensa oral pesa 50 %. Por eso este informe no se limita a listar comandos: explica *por qué* cada uno.

2. Entorno y setup

Antes de tocar el repositorio hay que tener resueltas dos cosas: la identidad con la que van a quedar firmados los commits, y cómo se autentica Git contra GitHub. Desde 2021 GitHub no acepta la contraseña de la cuenta desde la terminal, así que hace falta un token o —el camino que usé— el GitHub CLI, que además deja Git configurado.

```
$ git config --global user.name
marcosdon28
$ git config --global user.email
marcosdon28@gmail.com
$ gh auth status
github.com
  ✓ Logged in to github.com account marcosdon28 (keyring)
  - Git operations protocol: ssh
$ ssh -T git@github.com
Hi marcosdon28! You've successfully authenticated...
```

La autenticación es por **SSH**: la clave de esta máquina está cargada en la cuenta, así que `git push` no pide credenciales en cada operación. El protocolo elegido explica por qué los mensajes de error que aparecen más adelante dicen `github.com:marcosdon28/...` con dos puntos, y no `https://...`.

3. Creación del repositorio

Crear un repositorio es un acto de *administrador*, y ahí se toman tres decisiones:

- **Público** — requisito de la materia, y no por capricho: las funciones que se usan durante el semestre (protecciones, minutos de Actions ilimitados, environments con aprobaciones, secret scanning, CodeQL) son gratuitas *sólo* en repositorios públicos.
- **Con README** — así el repositorio nace con un archivo y con la rama `main` ya creada, en vez de nacer vacío.
- **El nombre** — `IngeSoft3-2319274`, que identifica materia y matrícula. Es el repositorio que se entrega en las dos presentaciones del año.

```
$ gh repo create IngeSoft3-2319274 --public --add-readme --clone
https://github.com/marcosdon28/IngeSoft3-2319274
```

4. El primer commit (y el único push directo del semestre)

El repositorio arranca con dos archivos, commiteados **antes** de activar la protección. Es el único momento del semestre en que un push directo a `main` va a funcionar, y queda declarado como tal.

`.gitignore`

Le dice a Git qué *no* se versiona. Arranca con el bloque base de la guía —artefactos de build, secretos, basura del sistema operativo y del editor— y le agregué desde ya los bloques de Python y Node, porque la app del semestre es FastAPI + React y esas carpetas (`__pycache__` / , `.venv` / , `node_modules` /) aparecen apenas empiece el TP2. La línea que más importa es `.env`: los secretos no se commitean nunca.

`CLAUDE.md`

Deja escritas las reglas de trabajo del repositorio: todo entra por PR, qué archivo se actualiza en qué momento, cómo se cierra cada TP con su tag. Es la misma idea que la protección de rama aplicada a las convenciones: **un proceso importante no debería depender de que alguien se acuerde**. Si está en un archivo, se lee, se revisa y se discute.

```
$ git add .gitignore CLAUDE.md
$ git commit -m "chore: agrega .gitignore base y CLAUDE.md con las reglas de trabajo del repo"
$ git push                                # funciona: la protección todavía no existe
```

5. Proteger `main`

Es la configuración más importante del trabajo práctico: hacer que **nadie** pueda pushear directo a `main` — ni siquiera yo, que soy el dueño. Todo cambio pasa a tener que entrar por un Pull Request.

La configuré por API en vez de por la web. Las tres tildes de la interfaz son más rápidas, pero un comando queda **reproducible y auditable**: se puede volver a correr, versionar y revisar. Es el mismo principio de *policy as code* que después sostiene los pipelines.

```
$ gh api --method PUT "repos/{owner}/{repo}/branches/main/protection" --input - <<'EOF'
{
  "required_pull_request_reviews": { "required_approving_review_count": 0 },
  "required_status_checks": null,
  "enforce_admins": true,
  "restrictions": null
}
EOF
```

Opción	Por qué está así
<code>required_pull_request_reviews</code>	Es la regla del juego: nada entra a <code>main</code> sin pasar por un PR.
<code>required_approving_review_count: 0</code>	Cero aprobaciones obligatorias. El TP es individual y GitHub nunca permite aprobar el propio Pull Request (por API devuelve <code>422 Can not approve your own pull request</code>). Con 1 aprobación no podría mergear nunca. En un equipo real acá iría 1 o más.
<code>enforce_admins: true</code>	Es el equivalente de <i>Do not allow bypassing</i> : la regla me alcanza también a mí. Sin esto GitHub me dejaría saltarla — y una protección que el dueño puede saltar es de adorno.
<code>required_status_checks: null</code>	Todavía no hay pipeline que exigir. En el TP4 este campo pasa a listar los checks de CI y el merge va a necesitar además el pipeline en verde.

marcosdon28 / IngeSoft3-2319274

Q Type to search

<> Code

Issues

Pull requests 1

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

General

Access

Collaborators

Moderation

Code, planning, and automation

Rulesets

Branches

Tags

Actions

Webhooks

Copilot

Planning

Environments

Codespaces

Pages

Security and quality

Advanced Security

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Autolink references

Branch protection rule

Level up your branch protections with Repository Rules

Rulesets are the recommended replacement for classic branch protections. They support rule layering, evaluate mode, and organization-wide policies. [Learn more about Rulesets.](#)

Branch name pattern *

main

Applies to 1 branch

main

Protect matching branches

☒ Require a pull request before merging

When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

☐ Require approvals

When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

☐ Dismiss stale pull request approvals when new commits are pushed

New reviewable commits pushed to a matching branch will dismiss pull request review approvals.

☐ Require review from Code Owners

Require an approved review in pull requests including files with a designated code owner.

☐ Require approval of the most recent reviewable push

Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

☐ Require status checks to pass before merging

Choose which [status checks](#) must pass before branches can be merged into a branch that matches this rule. When enabled, commits must first be pushed to another branch, then merged or pushed directly to a branch that matches this rule after status checks have passed.

☐ Require conversation resolution before merging

When enabled, all conversations on code must be resolved before a pull request can be merged into a branch that matches this rule. [Learn more about requiring conversation completion before merging.](#)

☐ Require signed commits

Commits pushed to matching branches must have verified signatures.

☐ Require linear history

Prevent merge commits from being pushed to matching branches.

☐ Require deployments to succeed before merging

Choose which environments must be successfully deployed to before branches can be merged into a branch that matches this rule.

☐ Lock branch

Branch is read-only. Users cannot push to the branch.

☒ Do not allow bypassing the above settings

The above settings will apply to administrators and custom roles with the "bypass branch protections" permission.

Rules applied to everyone including administrators

☐ Allow force pushes

Permit force pushes for all users with push access.

☐ Allow deletions

Allow users with push access to delete matching branches.

Save changes

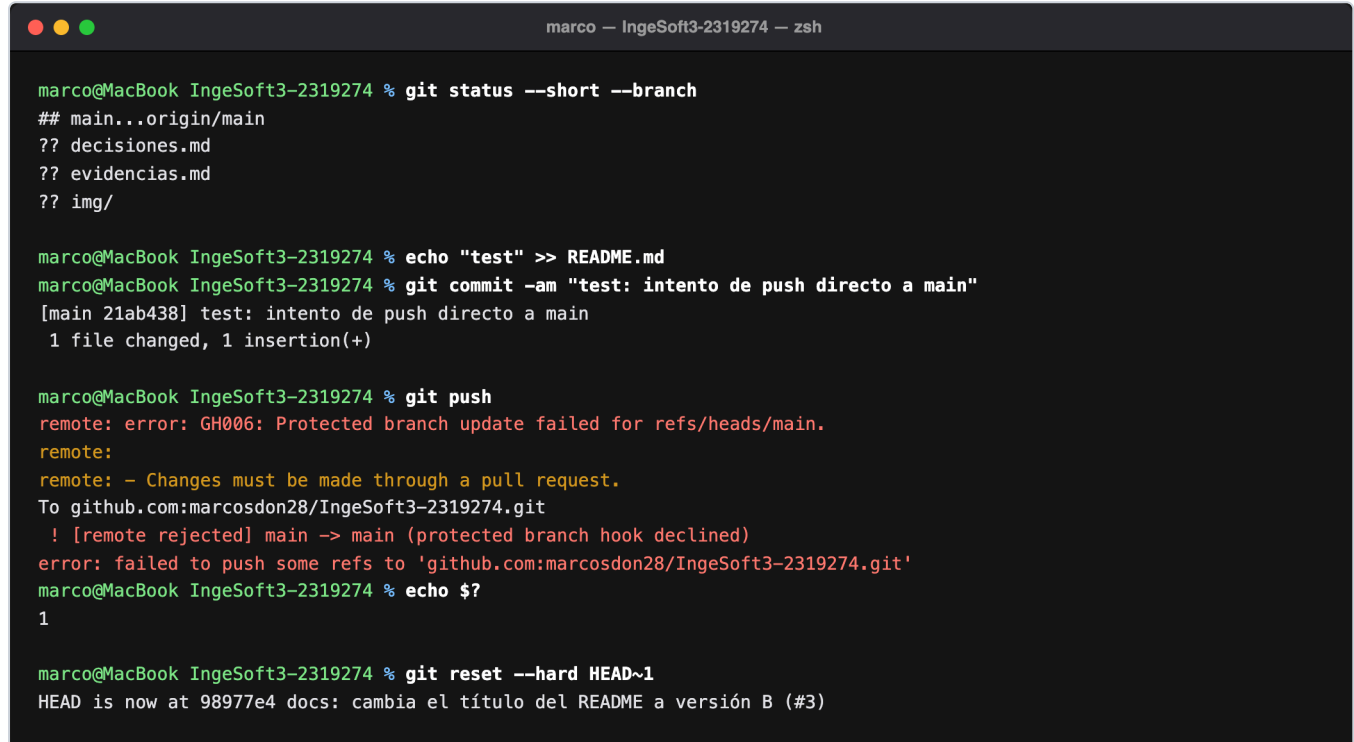
© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

Settings → Branches → regla sobre `main` . «Require a pull request before merging» tildado, «Require approvals» sin tildar (cero aprobaciones, deliberado), y «Do not allow bypassing the above settings» tildado.

6. La prueba de fuego: que la protección me rechace a mí

Una protección que nunca rechazó nada no se sabe si funciona. La única forma de verificarla es intentar violarla.

```
$ echo "test" >> README.md
$ git commit -am "test: intento de push directo a main"
$ git push                                # ← esto TIENE que fallar
```



```
marco@MacBook IngeSoft3-2319274 % git status --short --branch
## main...origin/main
?? decisiones.md
?? evidencias.md
?? img/

marco@MacBook IngeSoft3-2319274 % echo "test" >> README.md
marco@MacBook IngeSoft3-2319274 % git commit -am "test: intento de push directo a main"
[main 21ab438] test: intento de push directo a main
1 file changed, 1 insertion(+)

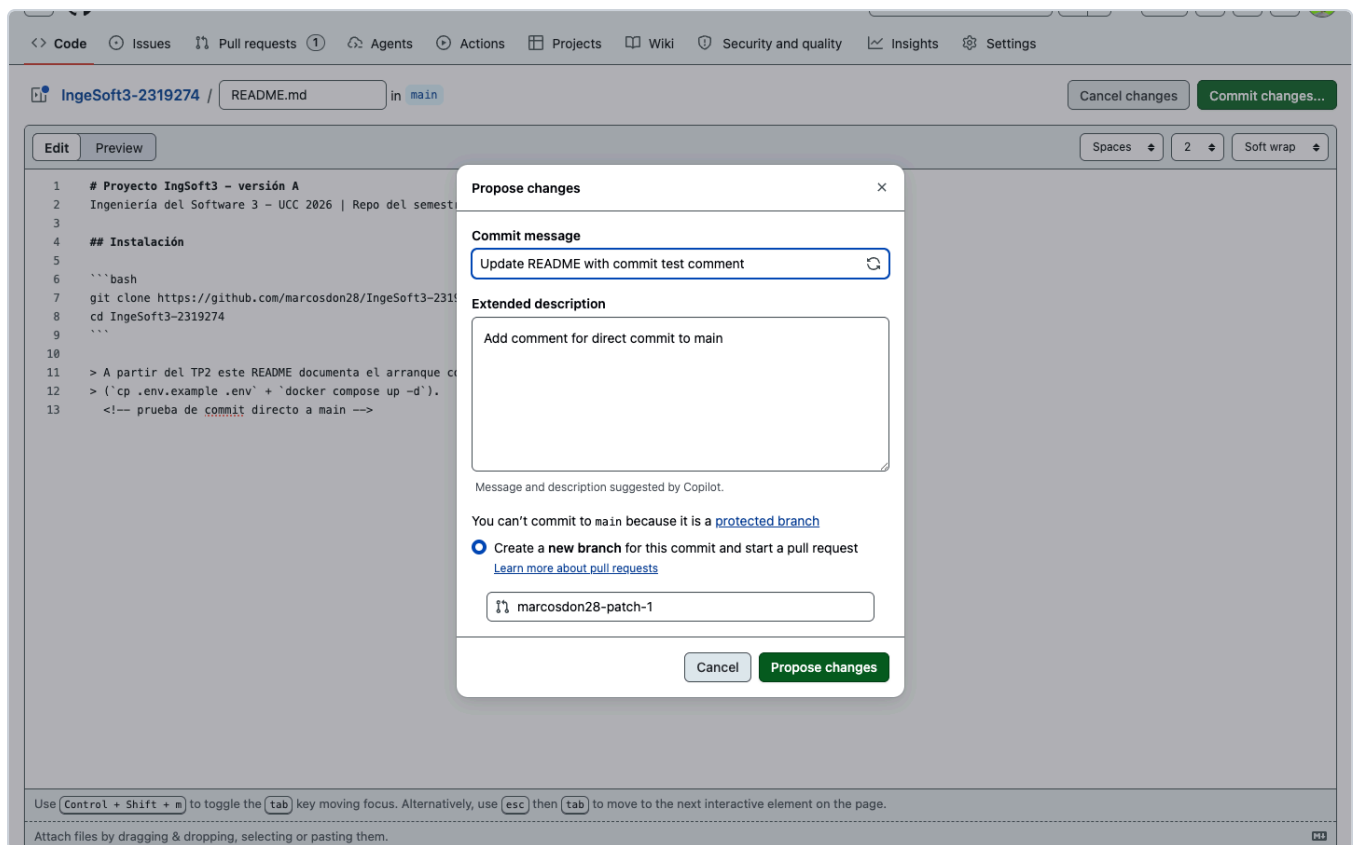
marco@MacBook IngeSoft3-2319274 % git push
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote:
remote: - Changes must be made through a pull request.
To github.com:marcosdon28/IngeSoft3-2319274.git
 ! [remote rejected] main -> main (protected branch hook declined)
error: failed to push some refs to 'github.com:marcosdon28/IngeSoft3-2319274.git'
marco@MacBook IngeSoft3-2319274 % echo $?
1

marco@MacBook IngeSoft3-2319274 % git reset --hard HEAD~1
HEAD is now at 98977e4 docs: cambia el título del README a versión B (#3)
```

Salida real del intento de push directo. GitHub responde `GH006: Protected branch update failed` y `! [remote rejected] main -> main (protected branch hook declined)`; el comando sale con código 1. Después, `git reset --hard HEAD~1` deshace el commit local que ya no sirve.

Dónde se rechaza, y por qué importa: el mensaje empieza con `remote: .` El commit se creó sin problema en mi máquina — Git local no sabe nada de políticas de GitHub. Lo que falló fue la *publicación*: el servidor evaluó la regla y se negó. Local y remoto son dos repositorios independientes, y la protección vive en uno solo de los dos.

La misma política, vista desde el navegador: intentando editar el README desde la web, GitHub avisa que no se puede commitear a `main` y ofrece la única salida posible.



«You can't commit to `main` because it is a protected branch». La única opción habilitada es *Create a new branch for this commit and start a pull request*: la regla no grita, desvía.

7. El ciclo del Pull Request

Este es el ciclo que se repite todo el semestre: rama corta → cambio → PR → lectura del diff → merge → borrar la rama → traer el cambio a la máquina.

```
$ git switch -c feature/seccion-instalacion
# ... editar el README ...
$ git add README.md
$ git commit -m "docs: agrega sección de instalación al README"
$ git push -u origin feature/seccion-instalacion
$ gh pr create --title "..." --body "..."
$ gh pr diff 1 # leer el cambio antes de integrarlo
$ gh pr merge 1 --squash --delete-branch
$ git switch main && git pull # ← el paso que siempre se olvida
```

Por qué *squash*

Las tres estrategias reales del botón de merge son *merge commit*, *squash* y *rebase*. Elegí *squash*: aplasta todos los commits de la rama en uno solo sobre `main`, así el historial queda lineal y legible —un commit por PR— y revertir un cambio completo es un solo `git revert`. Lo que se pierde es el paso a paso interno de la rama, que en ramas cortas no aporta gran cosa. El *fast-forward*, aclaración fina, no es una opción que se elija: es lo que pasa automáticamente cuando `main` no avanzó.

El último comando no es decorativo

El merge ocurrió **en GitHub**. Mi copia local no se enteraba sola: hasta que no hago `git pull`, mi `main` sigue en el commit viejo. Saltearse ese paso es la causa número uno del próximo conflicto tonto — y es exactamente lo que se fabrica a propósito en la sección que sigue.

8. Provocar y resolver un conflicto

Un conflicto no es un error: es lo que pasa cuando dos personas tocan la misma línea. Trabajando solo hay que fabricarlo, y conviene: es mucho mejor que el primer conflicto sea en un entorno controlado y no en el primer trabajo.

La receta

Dos ramas que nacen **las dos del mismo commit de main** y cambian la **misma** primera línea del README. El detalle donde se arruina el ejercicio es la base de la segunda rama: si B nace de A, ya tiene el cambio de A adentro y el merge entra limpio. Por eso nombré la base explícitamente en vez de confiar en dónde estaba parado.

```
$ git switch -c feature/titulo-a main # → "# Proyecto IngSoft3 - versión A"
$ git switch -c feature/titulo-b main # ← la base va explícita: main, no la rama A
                                     # → "# Proyecto IngSoft3 - versión B"
$ gh pr merge 2 --squash --delete-branch # A entra limpio...
```

The screenshot shows a GitHub Pull Request (PR) titled "docs: título del README - versión B #3" by user marcosdon28. The PR is open and shows a conflict in the README file. The comment from marcosdon28 states: "Cambia la MISMA primera línea del README que el PR #2, pero a # Proyecto IngSoft3 - versión B. Esta rama nació de main, sin conocer el cambio de la rama A. Cuando el PR #2 se mergee, este va a entrar en conflicto: dos ramas modificaron la misma línea y Git no puede decidir cuál vale." Below the comment, a commit titled "docs: cambia el título del README a versión B" is shown. A conflict warning box states: "This branch has conflicts that must be resolved. Use the web editor or the command line to resolve conflicts before continuing." The "Squash and merge" button is disabled. The right sidebar shows the PR details, including reviewers, assignees, labels, projects, milestones, and notifications.

El PR de la rama B después de mergear la A: cartel «Merge conflicts», el aviso «This branch has conflicts that must be resolved» con README.md señalado, y el botón *Squash and merge* deshabilitado.

Por qué Git no puede resolverlo solo

Git fusiona automáticamente cuando los cambios tocan partes distintas: compara ambas puntas contra el ancestro común y aplica los dos. Acá no puede: la misma línea cambió de dos formas incompatibles. Git tiene toda la información del problema y **ninguna para resolverlo** — cuál versión es «la correcta» no es

una pregunta técnica, es una decisión de contenido. Elegir por su cuenta significaría descartar en silencio el trabajo de alguien. Por eso marca el archivo y delega.

The screenshot shows a GitHub pull request interface for repository 'marcosdon28 / IngeSoft3-2319274'. The pull request is titled 'docs: título del README - versión B #3' and shows a conflict between 'feature/titulo-b' and 'main'. The conflict resolution tool is open for 'README.md'. On the left, a list of files shows 'README.md' with a conflict icon. The main area displays the conflict resolution tool with three tabs: 'Accept current change', 'Accept incoming change', and 'Accept both changes'. The current change (feature/titulo-b) is highlighted in yellow and shows lines 1-17 of the README.md file. The incoming change (main) is shown in grey. The conflict is located at line 2, where the current change has '# Proyecto IngSoft3 - versión B' and the incoming change has '# Proyecto IngSoft3 - versión A'. The tool provides options to accept either change or both. The 'Mark as resolved' button is disabled.

El editor de conflictos: <<<<<< feature/titulo-b (Current change) abre mi versión, ===== separa, >>>>>> main (Incoming change) cierra la que ya está en main. Arriba a la derecha, «1 conflict» y *Mark as resolved* deshabilitado.

Cómo lo resolví

Con una **síntesis**, no con una de las dos versiones. Los sufijos «versión A» y «versión B» existían sólo para fabricar el conflicto: quedarme con cualquiera de los dos habría dejado el README del semestre con un título de ejercicio. El título final quedó `# Proyecto IngSoft3 – Inventario de productos (UCC 2026)`. Ése es el punto del ejercicio: resolver es decidir contenido, y a veces el contenido correcto no es ninguna de las dos ramas.

```
$ git switch feature/titulo-b
$ git merge origin/main          # ← acá aparece el conflicto, con sus marcadores
# ... editar el archivo, dejar el contenido final, borrar los tres marcadores ...
$ git add README.md
$ git commit -m "fix: resuelve el conflicto de título del README con una síntesis"
$ git push
$ gh pr merge 3 --squash --delete-branch
```

Qué habría evitado que apareciera

1. **Integrar seguido.** Si antes de abrir la rama B hubiera hecho `git switch main && git pull`, B habría nacido del `main` que ya tenía el cambio de A: no hay dos versiones compitiendo, hay una sobre la otra. El conflicto no nace de tocar la misma línea, nace de *tocarla partiendo de estados distintos*.

2. **Ramas cortas.** Cuanto más vive una rama, más se aleja de `main` y más superficie de choque acumula. Es lo que DORA correlaciona con alto rendimiento: integrar a trunk al menos una vez por día.
3. **Trabajo repartido.** Si dos personas necesitan tocar la misma línea al mismo tiempo, suele ser señal de que las tareas no estaban bien separadas.

9. Versionar la entrega: tag y release

Un **tag** marca un commit con un nombre inmutable; una **release** le agrega la comunicación de qué cambió, escrita para humanos.

```
$ git tag -a v1.0.0 -m "TP1 cerrado - Git colaborativo"
$ git push origin v1.0.0           # los tags NO viajan con un push común
$ gh release create v1.0.0 --title "v1.0.0 - TP1: Git colaborativo" --notes "..."
```

The screenshot shows the GitHub interface for a repository named 'marcosdon28 / IngeSoft3-2319274'. The 'Releases' tab is selected, showing a list of releases. The latest release, 'v1.0.0 — TP1: Git colaborativo', is highlighted. The release notes are visible, detailing the first stable version of the semester and listing features such as a public repository with .gitignore, protection of the main branch, and resolved merge conflicts. The assets section shows two files: 'Source code (zip)' and 'Source code (tar.gz)', both uploaded 'now'.

La release publicada, con la etiqueta *Latest*, el tag `v1.0.0`, el commit que marca y las notas de qué incluye esta versión.

Qué significa el número

Versionado semántico: `MAJOR.MINOR.PATCH`. **MAJOR** sube cuando se rompe compatibilidad; **MINOR**, cuando se agrega funcionalidad compatible; **PATCH**, cuando se corrige un bug sin cambiar el comportamiento esperado. Acá es `1.0.0` porque es la primera versión estable: no hay compatibilidad previa que romper, ni funcionalidad agregada sobre algo anterior, ni bug corregido. Es el punto de partida.

El número de versión no es decorativo: es **información para el que consume** el software. En el TP2 la misma disciplina se aplica a las imágenes de contenedor, y en el TP6 los deploys nacen de tags.

El tag se movió, y queda declarado. Siguiendo el orden de la guía, la release se publicó antes de agregar `decisiones.md` y `evidencias.md` —que son justamente los archivos que documentan esa release—. Como el reglamento dice que el tag es «el punto exacto que se mira de cada TP», moví `v1.0.0` al commit final del trabajo con `git tag -f v1.0.0 && git push -f origin v1.0.0`, que es el procedimiento que el propio reglamento contempla.

10. Los entregables, y dónde está cada cosa

Qué	Dónde
URL del repositorio público	<code>github.com/marcosdon28/IngeSoft3-2319274</code>
Justificación de cada decisión + uso de IA	<code>decisiones.md</code> (raíz)
Las capturas con su explicación	<code>evidencias.md</code> + <code>img/</code>
Reglas de trabajo del repositorio	<code>CLAUDE.md</code> (raíz)
Este informe	<code>informes/TP1-informe.pdf</code> (fuente en <code>.html</code>)
Historial: 3 PRs con su diff y su conversación	pestaña <i>Pull requests</i> , cerrados
Estado congelado del TP	tag y release <code>v1.0.0</code>

Los tres Pull Requests mergeados, todos con estrategia squash y con la rama borrada después:

#	Qué hizo	Particularidad
1	Sección de instalación en el README	Primer PR: valida el flujo con la protección recién puesta
2	Título del README, versión A	Entró limpio
3	Título del README, versión B	Requirió resolver un conflicto de merge

Dos archivos que existen y no se commitean

`aprendizajes.md` y `notas-defensa-p1.md` son notas personales de estudio, no parte del entregable. Están ignorados vía `.git/info/exclude` y no vía `.gitignore`: el primero es el ignore **privado de este clon** y no viaja a quien clone el repositorio, mientras que el segundo se versiona y sí viaja. La contrapartida asumida es que `.git/info/exclude` no se clona, así que en otra máquina hay que rehacerlo — queda anotado en `CLAUDE.md`.

11. Problemas encontrados y aprendizajes

El orden de los pasos no es opcional

El `.gitignore` y el `CLAUDE.md` se commitearon antes de activar la protección. Queda entonces **exactamente un push directo a `main`** en todo el historial, y está declarado. Si la protección se activa primero, ni el commit inicial puede entrar sin PR: es correcto, pero hay que saberlo antes, no descubrirlo.

Al fabricar el conflicto, casi lo arruino

La rama B tiene que salir de `main`, no de la A. Nombrar la base explícitamente (`git switch -c feature/titulo-b main`) evita depender de dónde estaba parado.

GitHub no sabe al instante si un PR tiene conflicto

Apenas mergeada la rama A, consultar el estado del PR de B devolvía `mergeable: UNKNOWN`. No estaba roto: GitHub recalcula la mergeabilidad de forma asíncrona. Unos segundos después pasó a `CONFLICTING / DIRTY`. Es el mismo motivo por el que, después de resolver, a veces hay que esperar antes de poder mergear.

`sed -i` en macOS no es el de Linux

Pide un argumento de sufijo de backup: `sed -i '' '1s/.*\/...\/' README.md`, con las comillas vacías. Sin ellas, macOS interpreta el patrón como nombre de archivo de backup y falla con un mensaje que no menciona el problema real.

Verificar «existe» no es verificar «vale lo que espero»

Para automatizar las capturas del navegador, el chequeo de sesión iniciada miraba si existía el `<meta name="user-login">` de GitHub. Existe **también cuando estás deslogueado**, con el atributo vacío — así que el chequeo daba positivo siempre y las primeras capturas salieron con los botones *Sign in / Sign up* y sin el cuadro de merge, que GitHub no le muestra a un visitante anónimo. Se arregló exigiendo que el contenido no esté vacío. La lección excede a la herramienta: una aserción débil da falsos verdes, que es peor que no tener aserción.

No pude capturar una ventana de terminal con el sistema

El proceso desde el que trabajo no tiene el permiso de *Grabación de pantalla* de macOS: `screencapture` devuelve el fondo de escritorio sin ninguna ventana. En vez de dejar la evidencia incompleta, la salida real del push rechazado quedó transcrita literalmente en `evidencias.md` además de renderizada como imagen, y se agregaron dos evidencias equivalentes tomadas del navegador. Las tres muestran la misma política desde ángulos distintos.

Declaración de uso de Inteligencia Artificial

Usé **Claude Code (Claude Opus)** como asistente durante todo el TP1. El reglamento lo permite y lo alienta, con tres condiciones: declararlo, verificarlo y poder defenderlo.

Qué fue asistido	Cómo lo verifiqué
Ejecución de los comandos de <code>git</code> y <code>gh</code>	Leí cada comando antes de que corriera y comprobé su efecto en la web: el repositorio existe y es público, los PRs figuran mergeados con su diff, las ramas están borradas, el tag aparece en <i>Releases</i> .
Configuración de la protección por API	La leí de vuelta con <code>gh api ... --jq</code> y la comparé contra lo que muestra <i>Settings</i> → <i>Branches</i> . Y sobre todo, la probé : el push directo tiene que fallar, y falla.
Capturas de evidencia del navegador	Automatizadas con Playwright sobre este mismo repositorio y con mi sesión. Revisé cada imagen a ojo antes de aceptarla: descarté dos tandas, una deslogueada y otra con un tooltip tapando la interfaz. Las URLs son públicas y contrastables.
Redacción de <code>decisiones.md</code> , <code>evidencias.md</code> y este informe	Los revisé y corregí para que digan lo que efectivamente pasó. Los problemas listados son los que me pasaron a mí, no ejemplos genéricos.

Lo que **no** delegué: qué versión conservar al resolver el conflicto, la estrategia de merge, y el criterio de qué va commiteado y qué no.

Anexo — Los comandos, en orden

```
# — Repositorio —————
gh repo create IngeSoft3-2319274 --public --add-readme --clone

# — Primer (y único) push directo —————
git add .gitignore CLAUDE.md
git commit -m "chore: agrega .gitignore base y CLAUDE.md..."
git push

# — Ignore privado de este clon (no viaja al repo) —————
printf 'aprendizajes.md\nnotas-defensa-p1.md\n' >> .git/info/exclude

# — Protección de main —————
gh api --method PUT "repos/{owner}/{repo}/branches/main/protection" --input - <<'EOF'
{ "required_pull_request_reviews": { "required_approving_review_count": 0 },
  "required_status_checks": null, "enforce_admins": true, "restrictions": null }
EOF
gh api "repos/{owner}/{repo}/branches/main/protection" \
  --jq '{approvals: .required_pull_request_reviews.required_approving_review_count,
        admins: .enforce_admins.enabled}'

# — Prueba de fuego —————
echo "test" >> README.md
git commit -am "test: intento de push directo a main"
git push                                     # falla: protected branch hook declined
git reset --hard HEAD~1

# — PR #1 —————
git switch -c feature/seccion-instalacion
git add README.md && git commit -m "docs: agrega sección de instalación al README"
git push -u origin feature/seccion-instalacion
gh pr create --title "..." --body "..."
gh pr diff 1
gh pr merge 1 --squash --delete-branch
git switch main && git pull

# — El conflicto: PR #2 y #3 —————
git switch -c feature/titulo-a main          # base explícita
git commit -am "docs: cambia el título del README a versión A"
git push -u origin feature/titulo-a && gh pr create ...

git switch -c feature/titulo-b main          # base explícita otra vez
git commit -am "docs: cambia el título del README a versión B"
git push -u origin feature/titulo-b && gh pr create ...

gh pr merge 2 --squash --delete-branch        # A entra limpio
gh pr view 3 --json mergeable                 # → CONFLICTING

git switch feature/titulo-b
git merge origin/main                         # aparece el conflicto
# ... resolver, borrar los tres marcadores ...
git add README.md
git commit -m "fix: resuelve el conflicto de título del README con una síntesis"
```

```
git push
gh pr merge 3 --squash --delete-branch

# — Tag y release —————
git switch main && git pull
git tag -a v1.0.0 -m "TP1 cerrado - Git colaborativo"
git push origin v1.0.0
gh release create v1.0.0 --title "v1.0.0 - TP1: Git colaborativo" --notes "..."

# — Entregables, por PR como todo lo demás —————
git switch -c docs/entregables-tp1
git add decisiones.md evidencias.md img/ informes/
git commit -m "docs: agrega decisiones, evidencias e informe del TP1"
git push -u origin docs/entregables-tp1 && gh pr create ...

# — Congelar el estado final del TP —————
git tag -f v1.0.0 && git push -f origin v1.0.0
```